

# Multi-Zones Refactor Implementation Plan

---

**For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox ( - [ ] ) syntax for tracking.

**Goal:** Replace the `@module-federation/nextjs-mf` setup in `apps/host` + `apps/remote` with a native Next.js Multi-Zones architecture that matches the design in `docs/design-bff/mfe/`.

**Architecture:** `apps/host` becomes the **shell** (gateway + session + SSE proxy + telemetry). `apps/remote` is renamed to `apps/remote-app` and becomes **Zone 2** — a full Next.js app behind the shell's rewrite rules, with its own `basePath` / `assetPrefix`, BFF route, health check, and `_fragmento` endpoint. The shell and the zone never share a Webpack bundle; they communicate only over HTTP.

**Tech Stack:** Next.js (pages router, current version), pnpm workspaces, TypeScript.

---

## Global Constraints

---

- No new npm dependencies unless listed in a task. Remove `@module-federation/nextjs-mf`, `@module-federation/enhanced`, and the `webpack` override when the last task completes.
  - `exactOptionalPropertyTypes: true` must be in every zone's `tsconfig.json`.
  - Every cross-zone link must be a plain `<a>`, never `<Link>`.
  - Zone routes live under the zone's own prefix: `apps/remote-app` routes under `/remote-app`.
  - The shell NEVER contains a DAL or talks to any business domain directly.
  - Asset prefix for `apps/remote-app` must be `/remote-app-static`.
  - `basePath` for `apps/remote-app` must be `/remote-app`.
  - Zone health check responds without touching any domain: returns `{ ok: true }` only.
  - Fragment responses ( `/_fragmento/**` ) return only `text/html` with no `<script>` tags.
  - `204` is the only status for "not authorized" OR "nothing to show" — the consumer never knows which.
-

# File Map

---

## Deleted

- `apps/host/next.config.js` — entire Federation plugin block removed
- `apps/host/lib/safeRemoteLoader.ts` — Node-to-Node SSR fetch replaced by Multi-Zones redirect
- `apps/host/components/FederatedErrorBoundary.tsx` — Federation error boundary gone
- `apps/host/components/RemoteCardClientWrapper.tsx` — wrapper for federated lazy import, gone
- `apps/host/declarations.d.ts` — module declarations for `remote/*` types, gone

## Renamed

- `apps/remote/` → `apps/remote-app/` (all files move; only config files change substantially)

## Modified

- `pnpm-workspace.yaml` — strip Federation overrides (glob already covers `remote-app`)
- `apps/host/next.config.js` — replace Federation plugin with `async rewrites()` for Multi-Zones
- `apps/remote-app/next.config.js` — replace Federation plugin with `basePath` + `assetPrefix`
- `apps/host/pages/index.tsx` — remove lazy import of `remote/RemoteDashboard`; add `<a>` link to `/remote-app`
- `apps/host/components/HostLayout.tsx` — remove `isRemoteAvailable` + `onTabSelect` props
- `apps/host/components/SideNavigation.tsx` — cross-zone tabs become `<a>`
- `apps/remote-app/package.json` — `name` field: `remote` → `remote-app`

## Created

- `apps/remote-app/pages/api/health.ts` — health check, no domain I/O
  - `apps/remote-app/pages/_fragmento/[name]/[id].tsx` — fragment endpoint (GET, returns inert HTML or 204)
-

## Task 1: Rename `apps/remote` → `apps/remote-app` and update workspace

---

### Files:

- Modify: `pnpm-workspace.yaml`
- Modify: `apps/remote-app/package.json` (name field only)

### Interfaces:

- Consumes: nothing
- Produces: workspace recognizes `apps/remote-app`

- ☐ **Step 1: Rename the directory**

```
mv apps/remote apps/remote-app
```

- ☐ **Step 2: Update package name**

In `apps/remote-app/package.json`, change:

```
{ "name": "remote" }
```

to:

```
{ "name": "remote-app" }
```

- ☐ **Step 3: Re-install to refresh workspace symlinks**

```
pnpm install
```

Expected: no errors; both `apps/host` and `apps/remote-app` in the workspace.

- ☐ **Step 4: Commit**

```
git add apps/remote-app apps/host pnpm-workspace.yaml pnpm-lock.yaml
git commit -m "chore: rename apps/remote to apps/remote-app"
```

---

## Task 2: Configure `apps/remote-app` as a Multi-Zones zone

### Files:

- Modify: `apps/remote-app/next.config.js`
- Modify: `apps/remote-app/tsconfig.json`

### Interfaces:

- Consumes: directory from Task 1
- Produces: `remote-app` zone serves its pages under `/remote-app` with assets under `/remote-app-static`
- ☐ **Step 1: Write a test that the `basePath` is set**

Create `apps/remote-app/test/next-config.test.ts`:

```
// basePath and assetPrefix must match the shell's rewrite rules or assets 404
silently.
import nextConfig from '../next.config.js';

test('basePath is /remote-app', () => {
  expect(nextConfig.basePath).toBe('/remote-app');
});

test('assetPrefix is /remote-app-static', () => {
  expect(nextConfig.assetPrefix).toBe('/remote-app-static');
});
```

- ☐ **Step 2: Run it to confirm it fails**

```
cd apps/remote-app && npx tsx --test test/next-config.test.ts
```

Expected: FAIL — `nextConfig.basePath` is undefined (Federation config in place).

- ☐ **Step 3: Replace `apps/remote-app/next.config.js`**

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  basePath: '/remote-app',
  assetPrefix: '/remote-app-static',
};
```

```
module.exports = nextConfig;
```

- ☐ **Step 4: Add `exactOptionalPropertyTypes` to `tsconfig`**

In `apps/remote-app/tsconfig.json`, inside `compilerOptions`:

```
"exactOptionalPropertyTypes": true
```

- ☐ **Step 5: Run the test**

```
cd apps/remote-app && npx tsx --test test/next-config.test.ts
```

Expected: PASS.

- ☐ **Step 6: Start the zone and verify root page loads under `/remote-app`**

```
cd apps/remote-app && pnpm dev
```

Open `http://localhost:3001/remote-app` — should render the existing index page without 404.

- ☐ **Step 7: Commit**

```
git add apps/remote-app/next.config.js apps/remote-app/tsconfig.json apps/remote-app/test/
git commit -m "feat(remote-app): configure as Multi-Zones zone (basePath + assetPrefix)"
```

---

## Task 3: Reconfigure `apps/host` shell — rewrites + strip Federation

---

Files:

- Modify: `apps/host/next.config.js`
- Modify: `apps/host/tsconfig.json`
- Delete: `apps/host/declarations.d.ts`
- Modify: `apps/host/package.json` — remove `@module-federation/nextjs-mf`
- Modify: `apps/remote-app/package.json` — remove `@module-federation/nextjs-mf`

- Modify: `pnpm-workspace.yaml` — strip Federation overrides

## Interfaces:

- Consumes: zone running at `REMOTE_ZONE_URL` (default `http://localhost:3001`)
- Produces: shell at port 3000 proxies `/remote-app/**` and `/remote-app-static/**` to the zone

- ☐ **Step 1: Write routing tests**

Create `apps/host/test/rewrites.test.ts`:

```
// The rewrite config is the only routing source of truth.
// A missing rule means the zone 404s with no error — just a missing page.
import nextConfig from '../next.config.js';

test('rewrites export is a function', async () => {
  expect(typeof nextConfig.rewrites).toBe('function');
});

test('rewrites include zone root, sub-routes, and static assets', async () => {
  const rewrites = await nextConfig.rewrites!();
  const sources = (Array.isArray(rewrites) ? rewrites :
rewrites.beforeFiles ?? [])
    .map((r: { source: string }) => r.source);

  expect(sources).toContain('/remote-app');
  expect(sources).toContain('/remote-app/:path*');
  expect(sources).toContain('/remote-app-static/:path*');
});
```

- ☐ **Step 2: Run to confirm fail**

```
cd apps/host && npx tsx --test test/rewrites.test.ts
```

Expected: FAIL — current config has no `rewrites`.

- ☐ **Step 3: Replace `apps/host/next.config.js`**

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  async rewrites() {
    const remoteZoneUrl = process.env.REMOTE_ZONE_URL ?? 'http://
localhost:3001';
    return [
      // Zone root — separate from /:path* because /remote-app/:path* does NOT
      match /remote-app
```

```

    {
      source: '/remote-app',
      destination: `${remoteZoneUrl}/remote-app`,
    },
    // Zone sub-routes
    {
      source: '/remote-app/:path*',
      destination: `${remoteZoneUrl}/remote-app/:path*`,
    },
    // Zone static assets – separate prefix avoids /_next collision between
zones
    {
      source: '/remote-app-static/:path*',
      destination: `${remoteZoneUrl}/remote-app-static/:path*`,
    },
  ],
},
};

module.exports = nextConfig;

```

- ☐ **Step 4: Run the rewrite tests**

```
cd apps/host && npx tsx --test test/rewrites.test.ts
```

Expected: PASS.

- ☐ **Step 5: Delete Federation artifacts from host**

```
rm apps/host/declarations.d.ts
```

Remove `@module-federation/nextjs-mf` from `apps/host/package.json` `dependencies`.

- ☐ **Step 6: Strip Federation overrides from pnpm-workspace.yaml**

```

packages:
  - 'apps/*'

```

(Remove `allowBuilds`, `onlyBuiltDependencies`, and `overrides` blocks entirely.)

Also remove `@module-federation/nextjs-mf` from `apps/remote-app/package.json`.

- ☐ **Step 7: Re-install**

```
pnpm install
```

Expected: lockfile updated; no Federation packages installed.

- ☐ **Step 8: Commit**

```
git add apps/host/ apps/remote-app/package.json pnpm-workspace.yaml pnpm-lock.yaml
git commit -m "feat(host): replace Module Federation with Multi-Zones rewrites"
```

---

## Task 4: Rework `apps/host/pages/index.tsx` — remove federated lazy import

---

### Files:

- Modify: `apps/host/pages/index.tsx`
- Delete: `apps/host/components/FederatedErrorBoundary.tsx`
- Delete: `apps/host/components/RemoteCardClientWrapper.tsx`
- Delete: `apps/host/lib/safeRemoteLoader.ts`
- Modify: `apps/host/components/HostLayout.tsx`
- Modify: `apps/host/components/SideNavigation.tsx`

### Interfaces:

- Consumes: nothing from zone at runtime
- Produces: host index renders shell diagnostics and a plain `<a href="/remote-app">` for zone navigation

- ☐ **Step 1: Delete obsolete files**

```
rm apps/host/components/FederatedErrorBoundary.tsx
rm apps/host/components/RemoteCardClientWrapper.tsx
rm apps/host/lib/safeRemoteLoader.ts
```

- ☐ **Step 2: Rewrite `apps/host/pages/index.tsx`**

```
import React, { useState, useEffect } from 'react';
import type { GetServerSideProps, NextPage } from 'next';
import Head from 'next/head';
import HostLayout from '../components/HostLayout';
import { DEFAULT_SESSION, getSessionFromStorage, saveSessionToStorage, type
UserSession } from '../lib/session';
import { hostLog } from '../lib/logger';
```



```

interface HostHomePageProps {
  readonly hostRenderTimestamp: string;
  readonly initialSession: UserSession;
  readonly initialRoute: string;
}

const HostHomePage: NextPage<HostHomePageProps> = ({
  hostRenderTimestamp,
  initialSession,
  initialRoute,
}) => {
  const [currentSession, setCurrentSession] =
    useState<UserSession>(initialSession);

  useEffect(() => {
    const saved = getSessionFromStorage();
    if (saved.userId !== initialSession.userId) setCurrentSession(saved);
  }, [initialSession]);

  const handleSessionChange = (nextSession: UserSession) => {
    hostLog.client('HOST_SESSION_UPDATED', { user: nextSession.userName, role:
nextSession.role });
    setCurrentSession(nextSession);
    saveSessionToStorage(nextSession);
  };

  return (
    <>
      <Head>
        <title>Enterprise MFE Shell</title>
        <meta name="description" content="Next.js Multi-Zones shell – session,
SSE, telemetry" />
        <link rel="icon" href="/favicon.ico" />
      </Head>

      <HostLayout currentSession={currentSession}
onSessionChange={handleSessionChange}>
        <section className="host-section">
          <h2>Shell Runtime Diagnostics</h2>
          <p>SSR Rendered at: <strong>{hostRenderTimestamp}</strong></p>
          <p>Active route: <code>{initialRoute}</code></p>
        </section>

        <section className="host-section">
          <h2>Zones</h2>
          { /* Cross-zone navigation is always <a>, never <Link> – <Link> fails
silently across zones */ }
          <a href="/remote-app" className="zone-link"> Remote App Zone</a>
        </section>
      </HostLayout>
    </>
  );
};

export const getServerSideProps: GetServerSideProps<HostHomePageProps> = async
(context) => {

```

```

const hostRenderTimestamp = new Date().toISOString();
const session = DEFAULT_SESSION;
const initialRoute = context.resolvedUrl ?? '/';

hostLog.server('SSR_PAGE_RENDER', { route: initialRoute, user:
session.userName });

return { props: { hostRenderTimestamp, initialSession: session,
initialRoute } };
};

export default HostHomePage;

```

- ☐ **Step 3: Update** `apps/host/components/HostLayout.tsx`

Remove `isRemoteAvailable` and `onTabSelect` props — they only existed for the federated tab model:

```

'use client';

import React from 'react';
import Header from './Header';
import SideNavigation from './SideNavigation';
import ToastContainer from './ToastContainer';
import type { UserSession } from '../../lib/session';

interface HostLayoutProps {
  readonly children: React.ReactNode;
  readonly currentSession: UserSession;
  readonly onSessionChange: (session: UserSession) => void;
}

export const HostLayout: React.FC<HostLayoutProps> = ({ children,
currentSession, onSessionChange }) => (
  <div className="layout-root">
    <Header currentSession={currentSession} onSessionChange={onSessionChange} />
  >
    <div className="layout-body">
      <aside className="layout-sidebar"><SideNavigation /></aside>
      <main className="layout-main">{children}</main>
    </div>
    <ToastContainer />
  </div>
);

export default HostLayout;

```

- ☐ **Step 4: Update** `apps/host/components/SideNavigation.tsx`

Replace tab-based navigation with `<a>` links. Cross-zone links MUST be `<a>`, not `<Link>`:

```
import React from 'react';

// ponytail: static nav — add zone map config when zones > 2

export const SideNavigation: React.FC = () => (
  <nav className="side-nav">
    <a href="/" className="nav-item">Shell Home</a>
    { /* cross-zone: <a> required — <Link> soft-navigates and fails silently
across zones */ }
    <a href="/remote-app" className="nav-item">Remote App</a>
  </nav>
);

export default SideNavigation;
```

- ☐ **Step 5: Fix any TypeScript errors**

```
cd apps/host && npx tsc --noEmit
```

Fix any remaining type errors from removed props (Header, etc.).

- ☐ **Step 6: Start both apps and verify navigation**

Terminal 1: `cd apps/remote-app && pnpm dev` (port 3001)

Terminal 2: `cd apps/host && pnpm dev` (port 3000)

1. `http://localhost:3000` — shell home renders; no Federation errors in console.
2. Click “Remote App Zone” — browser hard-navigates to `http://localhost:3000/remote-app`.
3. Verify remote app content renders (proxied by shell rewrite).
4. DevTools Network: no `remoteEntry.js` requests.

- ☐ **Step 7: Commit**

```
git add apps/host/
git commit -m "feat(host): remove Module Federation; cross-zone navigation via
plain <a>"
```

---

## Task 5: Add health check and `_fragmento` endpoint to `apps/remote-app`

---

**Files:**

- Create: `apps/remote-app/pages/api/health.ts`

- Create: `apps/remote-app/pages/_fragmento/[name]/[id].tsx`

## Interfaces:

- Consumes: nothing (no domain I/O; ACL is stubbed for Rodada 1)
- Produces:
  - `GET /remote-app/api/health` → `{ ok: true }`, 200
  - `GET /remote-app/_fragmento/{name}/{id}` → inert `text/html` or 204
- ☐ **Step 1: Write health check test**

Create `apps/remote-app/test/health.test.ts`:

```
// Health check must respond without touching the domain.
// Domain downtime must NOT cause process restarts.
import type { NextApiRequest, NextApiResponse } from 'next';
import handler from '../pages/api/health';

type MockRes = { _status: number; _body: unknown; status: (n: number) => MockRes; json: (b: unknown) => void };

function mockRes(): MockRes {
  const res = {
    _status: 0,
    _body: undefined,
    status(n: number) { res._status = n; return res; },
    json(b: unknown) { res._body = b; },
  };
  return res;
}

test('returns 200 { ok: true }', () => {
  const res = mockRes();
  handler({} as NextApiRequest, res as unknown as NextApiResponse);
  expect(res._status).toBe(200);
  expect(res._body).toEqual({ ok: true });
});
```

- ☐ **Step 2: Run to confirm fail**

```
cd apps/remote-app && npx tsx --test test/health.test.ts
```

Expected: FAIL — file doesn't exist.

- ☐ **Step 3: Create** `apps/remote-app/pages/api/health.ts`

```
import type { NextApiRequest, NextApiResponse } from 'next';
```

```
// Process liveness check – no domain I/O.
// A check that queries the domain would restart healthy processes when the
domain is down.
export default function handler(_req: NextApiRequest, res: NextApiResponse):
void {
  res.status(200).json({ ok: true });
}
```

- ☐ **Step 4: Write fragment endpoint test**

Create `apps/remote-app/test/fragmento.test.ts`:

```
// Fragment contract (02-zonas.md §2):
// - GET only; 200 text/html OR 204; no <script> in response body
// - 204 covers both "not authorized" and "nothing to show" – consumer cannot
distinguish
import type { NextApiRequest, NextApiResponse } from 'next';
import handler from '../pages/_fragmento/[name]/[id]';

type MockRes = {
  _status: number; _body: string; _headers: Record<string, string>;
  status(n: number): MockRes;
  setHeader(k: string, v: string): MockRes;
  end(b?: string): void;
};

function mockRes(): MockRes {
  const r: MockRes = {
    _status: 0, _body: '', _headers: {},
    status(n) { r._status = n; return r; },
    setHeader(k, v) { r._headers[k] = v; return r; },
    end(b = '') { r._body = b; },
  };
  return r;
}

function mockReq(method: string, query: Record<string, string>):
NextApiRequest {
  return { method, query } as unknown as NextApiRequest;
}

test('GET returns 200 text/html for known fragment name', () => {
  const res = mockRes();
  handler(mockReq('GET', { name: 'demo', id: '1' }), res as unknown as
NextApiResponse);
  expect(res._status).toBe(200);
  expect(res._headers['Content-Type']).toMatch(/text\/html/);
  expect(res._body).not.toMatch(<script/i>);
});

test('GET returns 204 for unknown fragment name', () => {
  const res = mockRes();
  handler(mockReq('GET', { name: 'unknown', id: '1' }), res as unknown as
NextApiResponse);
});
```

```

    expect(res._status).toBe(204);
  });

  test('POST returns 405', () => {
    const res = mockRes();
    handler(mockReq('POST', { name: 'demo', id: '1' }), res as unknown as
NextApiResponse);
    expect(res._status).toBe(405);
  });

```

- ☐ **Step 5: Run fragment test to confirm fail**

```
cd apps/remote-app && npx tsx --test test/fragmento.test.ts
```

Expected: FAIL — file doesn't exist.

- ☐ **Step 6: Create** `apps/remote-app/pages/_fragmento/[name]/[id].tsx`

```

import type { NextApiRequest, NextApiResponse } from 'next';

// Known fragment names this zone exposes.
// {id} is encoded by the caller – never concatenated raw from client input.
// ponytail: ACL check stubbed – Rodada 1 is read-only. Wire
criarFragmento(@erp/nucleo) in Rodada 2.
const KNOWN_FRAGMENTS = new Set(['demo']);

export default function handler(req: NextApiRequest, res: NextApiResponse):
void {
  if (req.method !== 'GET') {
    res.status(405).end();
    return;
  }

  const { name, id } = req.query as { name: string; id: string };

  if (!KNOWN_FRAGMENTS.has(name)) {
    // 204 for both "not authorized" and "nothing to show" – consumer must not
distinguish them
    res.status(204).end();
    return;
  }

  const safeId = encodeURIComponent(String(id));

  res
    .status(200)
    .setHeader('Content-Type', 'text/html; charset=utf-8')
    // Fragment HTML is inert: no <script>, no inline handlers, no 'use
client'.
    // CSP nonce is per-request per-zone; a script from another zone would be
blocked anyway.
    .end(`<div class="fragment fragment--${name}"><p>Demo fragment (id: $

```

```
{safeId})</p></div>`);  
}
```

- ☐ **Step 7: Run all remote-app tests**

```
cd apps/remote-app && npx tsx --test test/health.test.ts test/fragmento.test.ts
```

Expected: all PASS.

- ☐ **Step 8: Commit**

```
git add apps/remote-app/pages/api/health.ts apps/remote-app/pages/_fragmento/  
apps/remote-app/test/  
git commit -m "feat(remote-app): add health check and _fragmento stub endpoint"
```

---

## Task 6: Final cleanup — verify end-to-end and remove last Federation traces

---

### Files:

- Any remaining `@module-federation` imports found by `grep`

### Interfaces:

- Consumes: everything from Tasks 1–5
- Produces: clean repo; both apps boot without Federation; all tests pass

- ☐ **Step 1: Grep for Federation remnants**

```
rg "@module-federation|remoteEntry|NextFederationPlugin|remote/ServerCard|  
remote/RemoteDashboard" apps/
```

Expected: zero matches. If any found, remove the import and the code that depends on it.

- ☐ **Step 2: Verify no `exposes` / `remotes` keys in any config**

```
rg "exposes:|remotes:" apps/
```

Expected: zero matches.

- ☐ **Step 3: Run all tests**

```
cd apps/host && npx tsx --test test/*.test.ts
cd apps/remote-app && npx tsx --test test/*.test.ts
```

Expected: all PASS.

- ☐ **Step 4: End-to-end smoke test**

Start both:

```
cd apps/remote-app && pnpm dev # port 3001
cd apps/host && pnpm dev # port 3000
```

| URL                                                                | Expected                                                     |
|--------------------------------------------------------------------|--------------------------------------------------------------|
| <code>http://localhost:3000/</code>                                | shell renders; no console errors                             |
| <code>http://localhost:3000/remote-app</code>                      | zone index renders via rewrite                               |
| <code>http://localhost:3000/remote-app/api/health</code>           | <code>{"ok":true}</code>                                     |
| <code>http://localhost:3000/remote-app/_fragmento/demo/42</code>   | 200, <code>text/html</code> , no <code>&lt;script&gt;</code> |
| <code>http://localhost:3000/remote-app/_fragmento/unknown/1</code> | 204                                                          |
| Network tab                                                        | zero <code>remoteEntry.js</code> requests                    |

- ☐ **Step 5: Final commit**

```
git add .
git commit -m "chore: complete Module Federation → Multi-Zones migration"
```

---



# Self-Review

## Spec coverage

| Doc requirement                                                                   | Task                                    |
|-----------------------------------------------------------------------------------|-----------------------------------------|
| Shell uses <code>rewrites()</code> for zone routing                               | Task 3                                  |
| <code>basePath: /remote-app</code> , <code>assetPrefix: /remote-app-static</code> | Task 2                                  |
| Root zone rewrite is a separate rule from <code>/:path*</code>                    | Task 3 (comment + test)                 |
| Cross-zone links are <code>&lt;a&gt;</code> , never <code>&lt;Link&gt;</code>     | Task 4 (index + SideNavigation)         |
| Zone health check, no domain I/O                                                  | Task 5                                  |
| <code>_fragmento</code> returns <code>text/html</code> or <code>204</code>        | Task 5                                  |
| Fragment HTML has no <code>&lt;script&gt;</code>                                  | Task 5 (test asserts it)                |
| <code>204</code> covers both “not authorized” and “nothing to show”               | Task 5 (comment + test)                 |
| <code>exactOptionalPropertyTypes: true</code> in zone tsconfig                    | Task 2                                  |
| Strip Federation plugin and webpack overrides                                     | Tasks 3 + 6                             |
| Shell has no DAL                                                                  | Task 4 (index page has no domain fetch) |

## Open (not in scope for this plan)

- **Redis session store + real ACL** — fragment stub skips ACL. Wire `criarFragmento(@erp/nucleo)` in Rodada 2 (flagged with `ponytail:` comment in Task 5).
- **SharedWorker for SSE** — Rodada 3.
- **Third zone** — Rodada 4.
- **`<Link>` -to-zone lint rule** — add when zone map is stable.